Server Temperature Monitoring and Predictive Maintenance Dashboard

Final project report

Name	Dharmapuri Srikrishna Ashrit
Registration number	235890216
Institution	Manipal Institute of Technology, Bengaluru
Date	21 October 2026

Live dashboard: https://server-temp-monitor.streamlit.app/
Source code (GitHub): https://github.com/KRISHNA2107DK/Server-Temperature-Monitoring-and-Predictive-Maintenance-Dashboard

The live dashboard is hosted on Streamlit Community Cloud (free tier). If the app has been idle, it may show a "wake up" button. Click it and wait about a minute for it to start.

Contents

1. Summary
2. Problem statement and intended users
3. Requirements and how each one is met
4. The dataset
5. Solution design and technology choices
6. Data processing: step by step
7. Analysis methods: overheating, abnormal patterns, prediction, risk levels
8. The dashboard: page-by-page guide
9. Key findings from the data
10. Testing and verification
11. Assumptions, limitations and data-quality notes
12. How to reproduce the results
13. Project files and deliverables
14. Conclusion

1. Summary

I built an interactive web dashboard that reads the real temperature logs collected from **14 laboratory servers** and checks every hardware component against the **critical limit of 85 °C** given in the problem statement. The dashboard answers four questions:

1. **Which components are overheating?** Every reading above 85 °C is counted, grouped into overheating events and shown in red.
2. **How are temperatures changing over time?** Time-series charts for every component of every server.

3. **Is anything behaving abnormally?** Sudden jumps or drops in temperature are detected automatically, even when they stay below 85 °C.
4. **What should be fixed first?** Every component gets a maintenance risk level (Critical, High, Warning or Normal), based on its past overheating, abnormal behaviour and current temperature trend.

The whole analysis was run on the full dataset: **1,376,036 time-stamped readings**, which contain **14,234,677 individual component temperatures**. All numbers in this report come from that run. None are estimated or invented.

2. Problem statement and intended users

Problem (in plain words): The lab's servers log the temperature of each of their hardware components roughly every 10 seconds. If any component goes above 85 °C, it may be developing a hardware fault, and the server may go down. Looking through tens of thousands of raw log files by hand is not practical. The lab needs one place where it can see the temperatures visually, spot overheating parts, follow trends, notice unusual behaviour and plan maintenance *before* a failure happens.

Intended users: lab engineers, system administrators and maintenance staff who look after the servers.

Typical workflow:

1. Open the dashboard link.
2. The **Overview** page immediately lists every component that went above 85 °C.
3. Pick a server on **Temperature trends** to see when and how often it overheated.
4. Check **Abnormal patterns** for sudden temperature jumps.
5. Use **Predictive maintenance** to see the ranked list of components to inspect first.
6. Use **Cooling & workload** to see whether fans or workload explain the heat.
7. Download any table as CSV for a maintenance ticket.

Example: the raw log line `2025-04-27 00:00:13, active, SDR0-FPGA = 56 °C, …` from server 172.27.186.134 appears on the trend chart as 56 °C for SDR0-FPGA, which is normal (below 80 °C). A reading of 87 °C would appear as a red ✕ above the dashed 85 °C line. It would be counted in "Minutes above 85 °C" and listed as a breach event, and the component would be rated High or Critical.

3. Requirements and how each one is met

Every requirement below comes directly from the wording of the problem statement. None was dropped or weakened.

#	Requirement (problem statement wording)	How it is met	Where to see it
1	Readings from *multiple servers*, each with *multiple components*	All 14 servers and all 17 temperature sensor types in the zip file are loaded automatically. Nothing is hard-coded.	Overview: Servers = 14, Server-components = 179
2	Critical threshold of *85 °C for any component*	The same 85 °C rule is applied to every component. "Exceeding" means strictly above 85 °C. Every temperature chart shows a red dashed 85 °C line.	All charts; `critical_breach_events.csv`

3	*Interactive* dashboard	A web dashboard with a top navigation bar and a filter bar (servers, components, active/idle status, date range). Charts can be zoomed and hovered for exact values, and legend entries can be clicked to hide or show lines.	Whole dashboard
4	*Multiple graphical formats*	Seven chart types: bar chart, heatmap, line chart (with time slider), box plot, grouped box plot, scatter plot, and a two-panel temperature-plus-fan chart. KPI cards and colour-coded tables add to these.	All pages
5	*Identify overheating components*	A red alert box names every server-component above 85 °C with its peak. The bar chart colours them red. A table lists every overheating event with start, end, duration and peak.	Overview; Abnormal patterns
6	*Monitor temperature trends*	A time-series line for each component of the chosen server. Minutes above 85 °C are marked, and a range slider allows zooming into any period.	Temperature trends
7	*Detect abnormal patterns*	An automatic statistical check flags any minute that differs sharply from the same component's previous hour (Section 7.2).	Abnormal patterns
8	*Proactively address potential hardware failures* (predictive maintenance)	A trend line is fitted to each component's recent data to project how many days remain until it reaches 85 °C. This is combined with past overheating and abnormal behaviour into four risk levels, with a "fix these first" list.	Predictive maintenance

4. The dataset

Item	Details (checked by reading the files)
Source file	`Tempreture Data.zip` (47 MB compressed, 209 MB uncompressed), provided with the assignment
Structure	`30th_April/<server-IP>_info/heat-monitor-logs/heat-monitor_<date>.csv`, one folder per server
Servers	14 (IP addresses 172.27.x.x)
CSV files	65,990 files: **131 contain readings**, 65,857 contain only the header row, 2 are completely empty. Empty and header-only files are skipped.
Columns (same in every file)	`Timestamp`, `STATUS` (active/idle), 16 SDR board sensors (`SDR0-7-FPGA`, `SDR0-7-RFIC`), `CPU-TEMP`, `CPUTIN`, `AUXTIN0-3`, `SYSTIN` (all in °C), and `FAN-1…7` (in rpm)
Missing sensors	Shown as "-" in the logs. They are treated as "sensor not present", not as zero.
Sampling	About one row every 10 seconds per server
Time span	Mostly 19 March 2025 to 30 April 2025. Server 172.27.186.50 also has 11,627 minutes dated 2–3 March 2023, most likely because that server's clock was wrong. These rows are kept and shown exactly as logged.

Size after loading	1,376,036 rows → 14,234,677 valid component temperatures → 3,185,417 one-minute summaries
Other files	`system_ip_info.txt` and `error_log.txt` (hardware descriptions and setup errors). These are not temperature data, so they are not used.

The 17 temperature sensors found in the data, and what they measure:

Sensor	Meaning
SDR0-FPGA … SDR4-FPGA	FPGA chip on software-defined-radio (SDR) board 0–4
SDR0-RFIC … SDR4-RFIC	RF integrated circuit (radio front-end chip) on SDR board 0–4
CPU-TEMP	Processor (CPU) die temperature
CPUTIN	CPU socket temperature, read by the motherboard sensor chip
SYSTIN	System / motherboard temperature
AUXTIN0 … AUXTIN3	Auxiliary motherboard sensors (e.g. voltage regulator or chipset area). Their exact location depends on the motherboard model, so this is an assumption.

SDR5–SDR7 columns exist in the files but contain no readings ("-") on any server, so they do not appear in the results.

5. Solution design and technology choices

5.1 Overall design

The project has two simple stages:

```
Tempreture Data.zip
      │  (1) src/pipeline.py — read, clean, summarise, detect, score   [run once, ~4 min]
      ▼
data/processed/*.parquet  +  results/tables/*.csv
      │  (2) src/dashboard.py — interactive web dashboard (Streamlit)
      ▼
Browser: https://server-temp-monitor.streamlit.app/
```

Why two stages? Reading 65,990 files and 14 million temperatures takes about four minutes and over 3 GB of memory. That is too slow and too large to repeat every time someone opens the dashboard. So the heavy work is done once, and the dashboard opens the small, pre-processed result files (about 31 MB) in seconds.

5.2 Options compared

Criterion	Python + Streamlit + Plotly (chosen)	Python + Plotly Dash	Power BI / Grafana
Meets all requirements	Yes	Yes	Yes
Easy to understand and explain	High: plain Python, about 760 lines of application code	Medium: needs callback code for every interaction	Low: logic hidden in a GUI tool
Setup effort	One `pip install` command	Same	Separate installers; Grafana also needs a database

Cost / accounts	Free; free public hosting on Streamlit Community Cloud	Free; hosting needs your own server	Power BI sharing needs a paid licence; Grafana needs a server
Reproducible from source code	Yes, everything is in the GitHub repository	Yes	Partly (binary report files)

5.3 What each component does, and why it was chosen

Component	Responsibility	Why it was chosen
Python 3.11	Programming language for everything	Standard for data analysis; already installed
pandas	Reading CSVs, cleaning, one-minute summaries, statistics	The standard table-processing library
pyarrow (Parquet files)	Saves the processed data in a compact, fast format	31 MB instead of hundreds of MB of CSV; loads in about 1 second
Plotly	All interactive charts (zoom, hover, legend toggle)	Interactive by default; the same chart code is used in the dashboard and for export
Streamlit	The web dashboard: navigation, filters, tables, downloads	A full web app in pure Python, with free one-click hosting
kaleido	Saves charts as PNG images for reports	Plotly's official image exporter
pytest	Automatic unit tests	The standard Python testing tool
GitHub + Streamlit Community Cloud	Code hosting and public deployment	Free, with no API keys or secrets needed

Not used, on purpose: no database (files are enough for a fixed dataset), no machine-learning model (there are no records of actual hardware failures to train one honestly; see Section 7.3), and no paid services or API keys.

6. Data processing: step by step

All steps are in `src/pipeline.py`. Each step is listed with the decision made and the reason for it.

#	Step	Decision and reason
1	Read the CSVs directly from the zip	No manual extraction is needed, so the result is always produced from the original file. Files smaller than 500 bytes (header only) are skipped because they contain no readings.
2	Take the server name from the folder	`172_27_186_13_info` becomes `172.27.186.13`, so every reading is tied to the correct server.
3	Clean column names	`SDR0-FPGA(°C)` becomes `SDR0-FPGA`. The files use an older text encoding (Latin-1) for the degree sign, so they are read with that encoding to avoid errors.
4	Treat "-" as missing	A "-" means the sensor is not fitted. Counting it as 0 °C would be wrong.
5	Remove duplicate rows	The same server and timestamp appearing twice (overlapping log files) is kept once.

6	Remove impossible values	Values below −20 °C or above 150 °C are treated as sensor glitches and excluded. This does not affect the 85 °C analysis because the limits are far outside the real range.
7	Separate fans from temperatures	Fan columns are in rpm, not °C, so they are kept out of the temperature analysis and shown separately on the Cooling page.
8	Summarise to one row per minute, keeping both the **average** and the **maximum**	Fourteen million points are too many for a browser. Keeping the maximum as well as the average means a short spike above 85 °C can never be hidden by averaging.
9	Attach server status per minute	The last `STATUS` (active/idle) logged in each minute is attached, so temperatures under load can be compared with idle temperatures.
10	Run the analysis (Section 7)	Overheating events, abnormal minutes, trend, risk level.
11	Save outputs	Processed data goes to `data/processed/`, result tables to `results/tables/` and the run log to `results/pipeline_log.txt`.

> **Important:** the overheating counts in the result tables and in the risk levels use the **original ~10-second readings**, not the one-minute summaries, so nothing is lost to averaging. The dashboard's "Minutes above 85 °C" counts minutes whose *maximum* reading was above 85 °C.

7. Analysis methods

7.1 Overheating (critical breaches)

- A reading is **critical** if it is **strictly greater than 85 °C**. A reading of exactly 85 °C is not counted, because the problem statement says "exceeding".
- Back-to-back critical readings of the same component form one **overheating event**. A gap of more than 5 minutes starts a new event. Each event records its start, end, duration, peak temperature and number of readings.
- An 80–85 °C **warning band** is shown as an early alert. This band is my own assumption; only 85 °C is given in the brief.

7.2 Abnormal patterns (anomaly detection)

- For each component, the average and spread (standard deviation) of its **previous 60 minutes** are calculated.
- A minute is **abnormal** if it is more than **4 standard deviations** away from that previous-hour average (a "rolling z-score" above 4).
- In simple words: a temperature that suddenly jumps or drops far more than it normally moves is flagged, even if it is still below 85 °C. This can reveal a failing fan, a sensor fault or a sudden workload change early.
- The spread has a minimum of 0.5 °C, so that a sensor that is almost perfectly flat does not get flagged for tiny 0.1 °C changes.
- The threshold of 4 was chosen to keep false alarms low. It is a setting in the code (`ANOMALY_Z`) and can be changed.

7.3 Prediction (temperature trend)

- For each component, a straight trend line is fitted to its **last 3 days of data** (least-squares linear regression). The slope gives the warming or cooling rate in **°C per day**.

- If the component is warming, the line is extended to estimate **days until it reaches 85 °C**. If it is already above 85 °C, this is 0. If it is flat or cooling, no breach is projected.
- A trend is only calculated when the recent data covers **at least one full day**. Projecting days ahead from a few minutes of warm-up data would give a meaningless result.
- **Why not machine learning?** The dataset has no record of real hardware failures, so there are no examples to train or test a failure-prediction model. A transparent trend line can be explained and checked by hand. A model that could not be validated would not be honest.

7.4 Maintenance risk level

The rules are checked in this order, and each component gets the first level that applies:

Level	Rule	Meaning for maintenance
CRITICAL	Latest temperature is above 85 °C, **or** at least 1% of all its readings were above 85 °C	Act now: overheating is ongoing or repeated
HIGH	At least one reading above 85 °C, **or** the trend reaches 85 °C within 7 days	Plan an inspection soon
WARNING	Peak of 80 °C or more, **or** at least one abnormal minute	Keep watching
NORMAL	None of the above	No action needed

8. The dashboard: page-by-page guide

Layout: a **navigation bar at the top** switches between six pages. Under the title, a **filter bar** (click to open) applies to every page: servers, components, server status (active/idle) and date range. Every chart sits on a white card with dark text and grid lines for high contrast. Legends sit in a grey box with a bold title. Each component keeps the same colour on every chart, and all charts are labelled with component names. The 85 °C line (red dashed) and 80 °C line (orange dotted) are drawn on every temperature chart.

Interaction on every chart: hover to see exact values, drag to zoom, double-click to reset, click a legend entry to hide or show it, and double-click a legend entry to show only that one.

8.1 Overview

- **Five KPI cards:** number of servers, server-components monitored, peak temperature, minutes above 85 °C, abnormal minutes.
- **Red alert box** naming every server-component that exceeded 85 °C, with its peak temperature.
- **Bar chart:** the 40 hottest server-components, coloured red (critical), amber (warning) or green (normal).
- **Heatmap:** every server (rows) against every component (columns), coloured by peak temperature, with the value written in each cell.
- **Server summary table** with a CSV download button.

Overview | Temperature trends | Abnormal patterns | Predictive maintenance | Cooling & workload | Component guide | Stop | Deploy

Server Temperature Monitoring & Predictive Maintenance

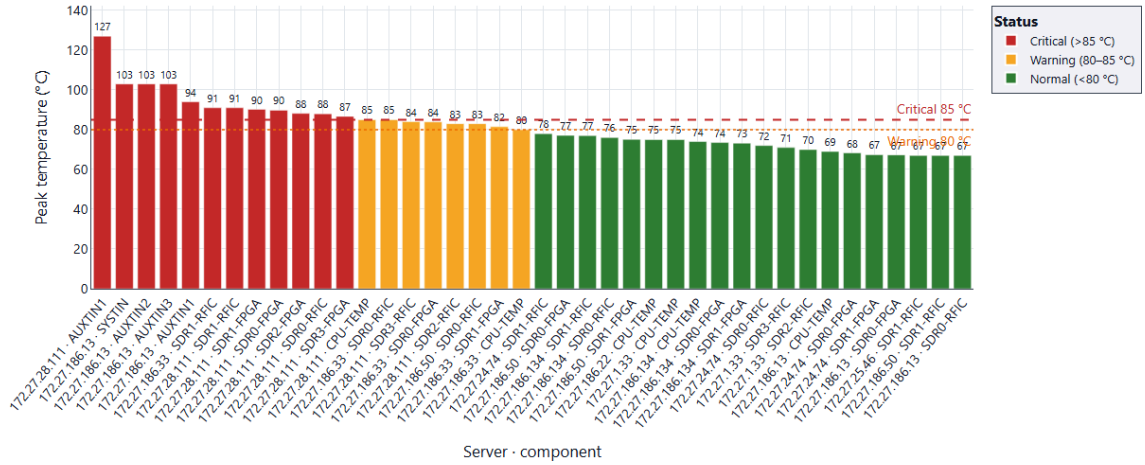
Critical threshold 85 °C · warning band 80–85 °C (assumption) · data = 1-minute mean/max of raw ~10 s readings from `Tempreture Data.zip`

Filters — servers, components, server status, dates

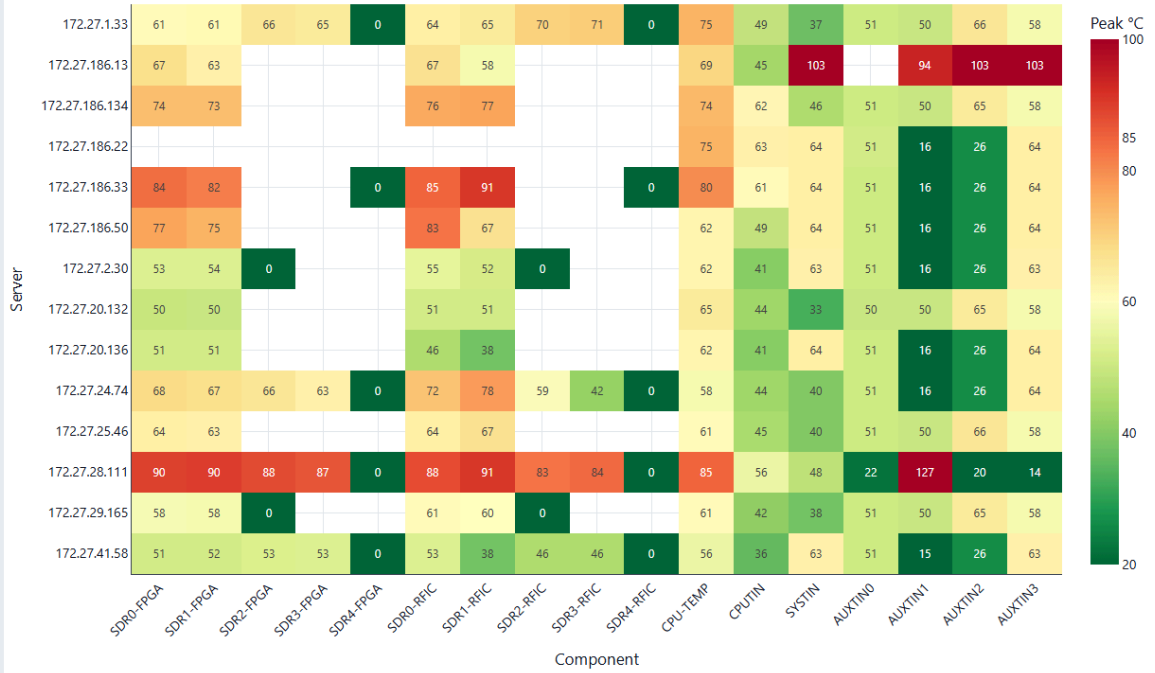
Servers	Server-components monitored	Peak temperature	Minutes above 85 °C	Anomalous minutes
14	179	127.0 °C	197,882	18,590

12 server-components exceeded 85 °C: **172.27.28.111 · AUXTIN1** (127 °C), **172.27.186.13 · AUXTIN2** (103 °C), **172.27.186.13 · SYSTIN** (103 °C), **172.27.186.13 · AUXTIN3** (103 °C), **172.27.186.13 · AUXTIN1** (94 °C), **172.27.186.33 · SDR1-RFIC** (91 °C), **172.27.28.111 · SDR1-RFIC** (91 °C), **172.27.28.111 · SDR1-FPGA** (90 °C), **172.27.28.111 · SDR0-FPGA** (90 °C), **172.27.28.111 · SDR2-FPGA** (88 °C), **172.27.28.111 · SDR0-RFIC** (88 °C), **172.27.28.111 · SDR3-FPGA** (87 °C)

Top 40 hottest server · component pairs (peak °C)



Peak temperature (°C) by server and component



Server summary

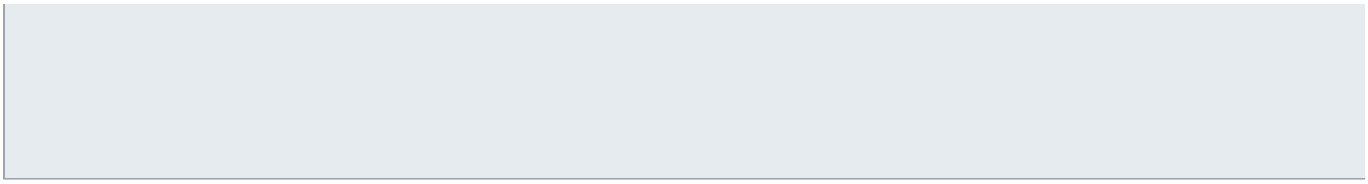


Figure 1 – Overview page (screenshot of the running dashboard, all filters at default)

8.2 Temperature trends

- Choose a server and its components. A **line chart** shows each component over time, with a range slider to zoom into any period. Readings above 85 °C are marked with red ×.
- To keep the browser fast, a very large selection is shown in 5-minute (or longer) averages. The chart title states the resolution. Each block keeps its **maximum**, so a spike above 85 °C is still marked.
- A **box plot** shows the typical range of each component type. The box covers the middle 50% of readings, the line inside it is the median, and the whiskers reach the most extreme readings within 1.5 × the box height. Readings of exactly 0 °C (boards switched off) are left out of this chart only.

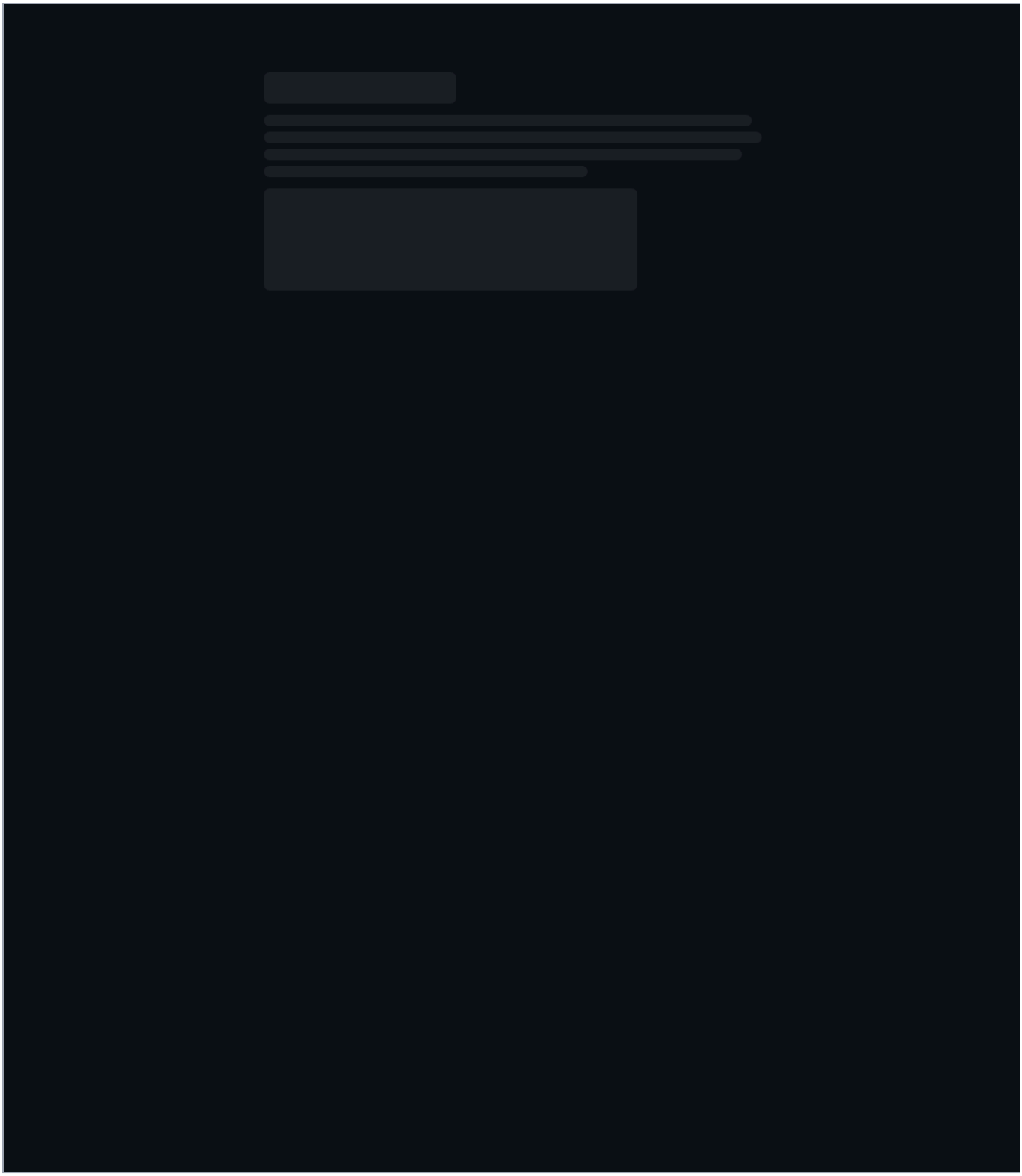


Figure 2 – Temperature trends page

8.3 Abnormal patterns

- A plain-language explanation of the anomaly rule.
- A **scatter plot** of every abnormal minute (colour = component, marker shape = server).
- A table of abnormal-minute counts per server-component, and the full table of **overheating events** (start, end, duration, peak), both downloadable as CSV.

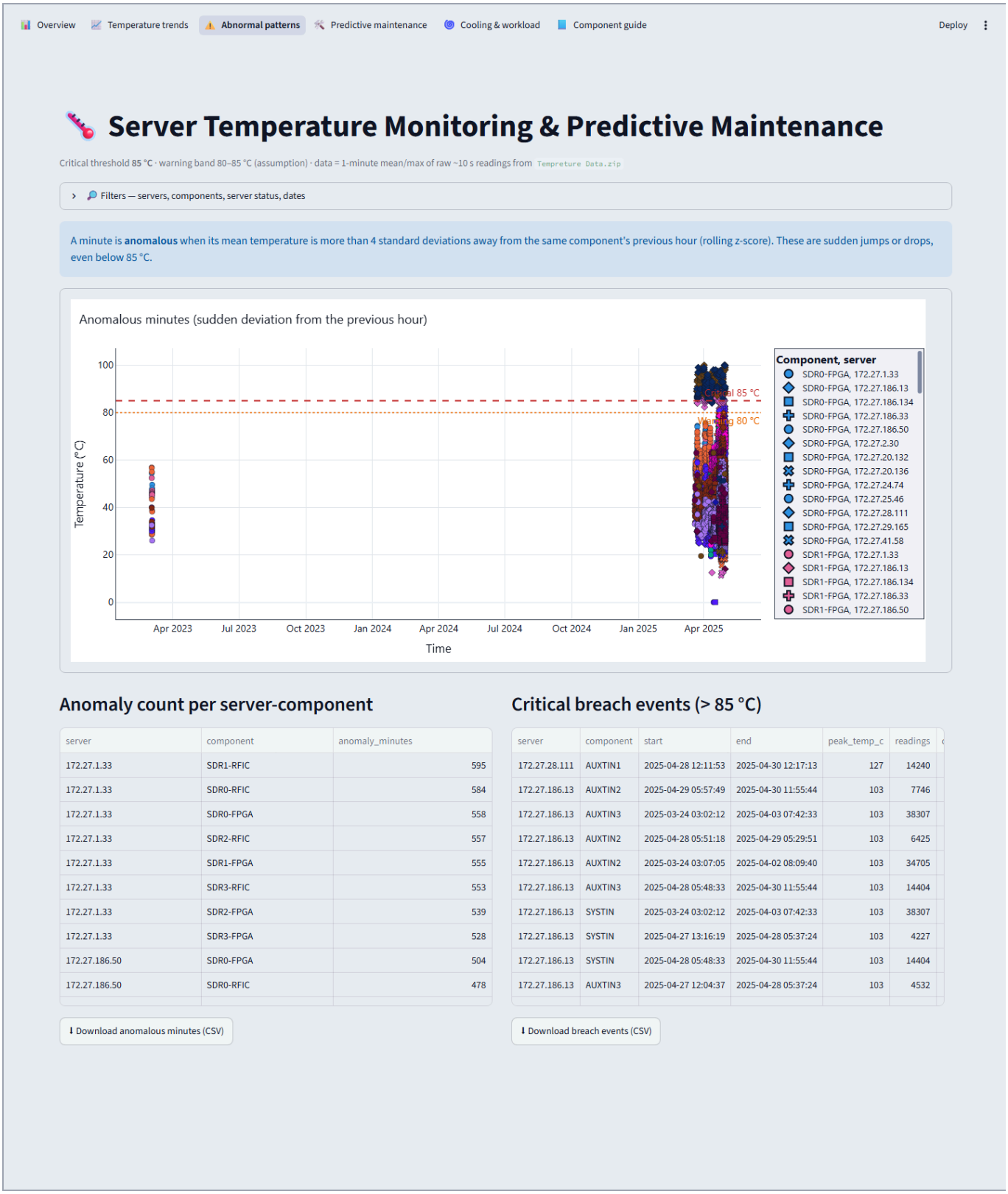
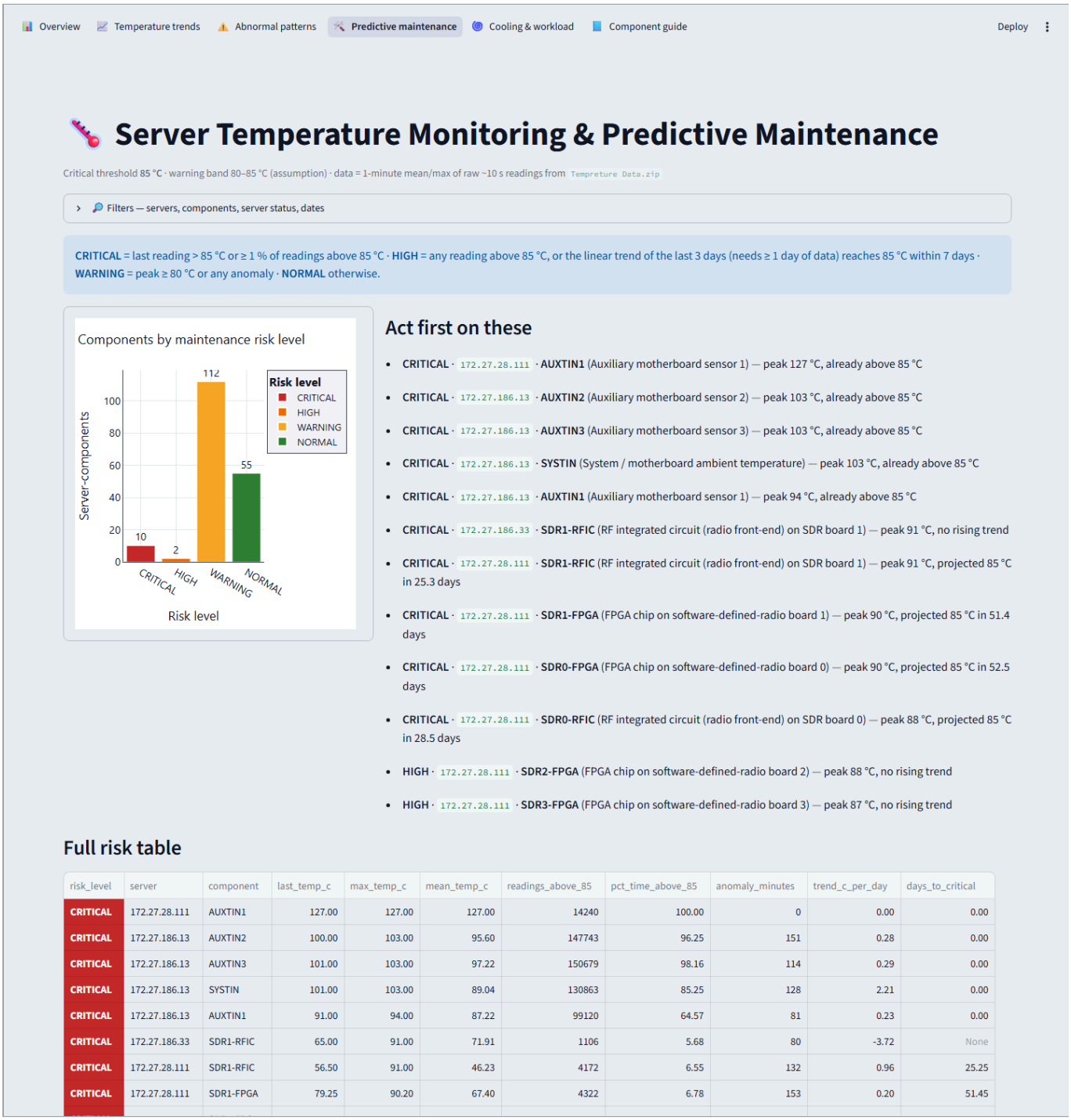


Figure 3 – Abnormal patterns page

8.4 Predictive maintenance

- The risk rules, written out on the page.
- A **bar chart** of how many components are at each risk level.
- An **"Act first on these"** list: every Critical and High component, with its server, a plain description of the component, its peak temperature and its projected time to 85 °C.

- The **full risk table** (colour-coded risk level, latest, peak and average temperature, readings above 85 °C, % of time above 85 °C, abnormal minutes, trend in °C/day, days to 85 °C), with a CSV download.



risk_level	server	component	last_temp_c	max_temp_c	mean_temp_c	readings_above_85	pct_time_above_85	anomaly_minutes	trend_c_per_day	days_to_critical
CRITICAL	172.27.28.111	AUXTIN1	127.00	127.00	127.00	14240	100.00	0	0.00	0.00
CRITICAL	172.27.186.13	AUXTIN2	100.00	103.00	95.60	147743	96.25	151	0.28	0.00
CRITICAL	172.27.186.13	AUXTIN3	101.00	103.00	97.22	150679	98.16	114	0.29	0.00
CRITICAL	172.27.186.13	SYSTIN	101.00	103.00	89.04	130863	85.25	128	2.21	0.00
CRITICAL	172.27.186.13	AUXTIN1	91.00	94.00	87.22	99120	64.57	81	0.23	0.00
CRITICAL	172.27.186.33	SDR1-RFIC	65.00	91.00	71.91	1106	5.68	80	-3.72	None
CRITICAL	172.27.28.111	SDR1-RFIC	56.50	91.00	46.23	4172	6.55	132	0.96	25.25
CRITICAL	172.27.28.111	SDR1-FPGA	79.25	90.20	67.40	4322	6.78	153	0.20	51.45

Figure 4 – Predictive maintenance page

8.5 Cooling & workload

- A **two-panel chart** for the chosen server: key temperatures on top and fan speeds (rpm) below, on the same time axis. A fan at 0 rpm while temperatures rise points to a cooling problem.
- A **fan summary table**: average, minimum and maximum rpm, and the % of time each fan was stopped.
- A **grouped box plot** comparing each component's temperature when the server is **active** (under load) and **idle**.

8.6 Component guide

A reference table explaining what every sensor measures, the colour it uses on every chart, and whether it appears in this dataset.

9. Key findings from the data

All figures below come from the full run recorded in `results/pipeline_log.txt` and the tables in `results/tables/`.

9.1 Overall

Measure	Result
Servers analysed	14
Server-component combinations monitored	179
Servers with at least one reading above 85 °C	**3 of 14**: 172.27.28.111, 172.27.186.13, 172.27.186.33
Server-components that exceeded 85 °C	12
Overheating events (consecutive readings above 85 °C)	237
Minutes with a reading above 85 °C	197,882
Abnormal minutes detected	18,590
Highest temperature recorded	127 °C (172.27.28.111, AUXTIN1; see the data-quality note in 9.3)
Risk levels (179 server-components)	Critical 10 · High 2 · Warning 112 · Normal 55 (total 179)

9.2 Components that need attention (Critical and High)

Level	Server	Component	Latest °C	Peak °C	Readings > 85 °C	% time > 85 °C	Abnormal min	Trend °C/day	Days to 85 °C
CRITICAL	172.27.28.111	**AUXTIN1** Auxiliary motherboard sensor 1	127.0	127.0	14,240	100.0	0	+0.00	0.0
CRITICAL	172.27.186.13	**AUXTIN2** Auxiliary motherboard sensor 2	100.0	103.0	147,743	96.3	151	+0.28	0.0
CRITICAL	172.27.186.13	**AUXTIN3** Auxiliary motherboard sensor 3	101.0	103.0	150,679	98.2	114	+0.29	0.0
CRITICAL	172.27.186.13	**SYSTIN** System / motherboard ambient temperature	101.0	103.0	130,863	85.3	128	+2.21	0.0
CRITICAL	172.27.186.13	**AUXTIN1** Auxiliary motherboard sensor 1	91.0	94.0	99,120	64.6	81	+0.23	0.0

CRITICAL	172.27.186.33	**SDR1-RFIC** RF integrated circuit (radio front-end) on SDR board 1	65.0	91.0	1,106	5.7	80	-3.72	–
CRITICAL	172.27.28.111	**SDR1-RFIC** RF integrated circuit (radio front-end) on SDR board 1	56.5	91.0	4,172	6.5	132	+0.96	25.3
CRITICAL	172.27.28.111	**SDR1-FPGA** FPGA chip on software-defined-radio board 1	79.2	90.2	4,322	6.8	153	+0.20	51.4
CRITICAL	172.27.28.111	**SDR0-FPGA** FPGA chip on software-defined-radio board 0	78.9	89.8	4,336	6.8	153	+0.20	52.5
CRITICAL	172.27.28.111	**SDR0-RFIC** RF integrated circuit (radio front-end) on SDR board 0	57.0	88.0	2,686	4.2	131	+0.86	28.5
HIGH	172.27.28.111	**SDR2-FPGA** FPGA chip on software-defined-radio board 2	73.3	88.2	7	0.0	114	-0.28	–
HIGH	172.27.28.111	**SDR3-FPGA** FPGA chip on software-defined-radio board 3	72.6	86.7	5	0.0	118	-0.25	–

"Days to 85 °C" = 0 means the component is already above 85 °C. "–" means no rise towards 85 °C is projected (flat or cooling trend). "Latest °C" is the average of the component's last 30 one-minute values.

9.3 What the findings mean

- **Server 172.27.186.13 is the most serious case.** Its motherboard sensors AUXTIN1–3 and SYSTIN were above 85 °C for 65–98% of all their readings, peaking at 103 °C. Its longest overheating event lasted about **10 days** (24 March to 3 April 2025). All five of its fan readings show **0 rpm for the whole period**. Either the fans have failed, or this board does not report fan speed. This should be checked physically, and is the first maintenance action I would recommend.
- **Server 172.27.28.111** has its SDR boards (SDR0–3, FPGA and RFIC chips) peaking at 87–91 °C. Four of them were above 85 °C for 4–7% of their readings, so its radio boards are running hot under load.
- **Data-quality note on 172.27.28.111 AUXTIN1:** this sensor reads **exactly 127 °C, unchanged, for its entire logged period** (28–30 April 2025). A perfectly constant 127 is the highest value an 8-bit temperature sensor can report, and usually means the sensor is faulty or not connected rather than a real temperature. The dashboard still flags it as Critical rather than hiding it, because a real 127 °C cannot be ruled out from the data alone. It should be checked on the hardware.
- **Server 172.27.186.33** had six short overheating events on its SDR1-RFIC chip (peak 91 °C). It is currently cooling (negative trend), but it overheated for 5.7% of its readings.

- **The other 11 servers never exceeded 85 °C.** Their hottest components stayed between 63 °C and 83 °C. They do show abnormal minutes. Across all servers, about **41%** of abnormal minutes (2,599 of 6,279 server-minutes; a server-minute is counted once even when several of its components are abnormal in it, which is why this is smaller than the 18,590 component-minutes) fall within 5 minutes of the server switching between idle and active. The rest are other sudden jumps or drops worth reviewing on the Abnormal patterns page.
- **Workload matters:** comparing median temperatures, the SDR FPGA chips run about **10–12 °C hotter**, the SDR RFIC chips about **8–29 °C hotter** and the CPU about **8 °C hotter** when servers are active than when idle (readings of exactly 0 °C excluded). This is shown on the Cooling & workload page.

10. Testing and verification

Check	Result
Automatic unit tests (`python -m pytest -q`)	17 passed, 1 warning in 1.24s (all tests passed)
What the tests cover	Server name and column name parsing; header-only files are skipped; "-" is treated as missing; the 85 °C rule is strictly "greater than"; consecutive readings are grouped into one event; the one-minute maximum keeps short spikes; fan and status extraction; the trend slope and days-to-85 °C calculation; no trend projection from less than one day of data; all four risk rules; a sudden jump is flagged as an anomaly; the box-plot numbers match the textbook definition; averaging into larger time blocks never loses the maximum. The tests use a small, hand-written sample file whose correct answers are known in advance. It is labelled as test data in the code.
Full pipeline on the real data	Completed. Log saved in `results/pipeline_log.txt`
Chart export	34 interactive HTML charts and 34 PNG images in `results/charts/`
Dashboard, all 6 pages, opened in a browser	No errors; the screenshots in Section 8 are from this check
Dashboard memory use (local measurement)	About 320 MB after opening every page, which is within Streamlit Community Cloud's documented range of 690 MB–2.7 GB
Consistency check	The dashboard KPI values (14 servers, 179 server-components, 127 °C peak, 197,882 minutes above 85 °C, 18,590 abnormal minutes) match the pipeline output exactly

11. Assumptions, limitations and data-quality notes

Assumptions (my own choices, not given in the brief)

1. "Exceeding 85 °C" means strictly above 85 °C.
2. The 80–85 °C warning band is an early-warning aid only.
3. The anomaly rule (more than 4 standard deviations from the previous hour) and the risk-level rules (Section 7.4) are reasonable, adjustable settings. They are not taken from any lab standard.
4. The descriptions of the CPUTIN, SYSTIN and AUXTIN sensors are the usual meanings of these motherboard sensor names. The exact board location of AUXTIN0–3 depends on the motherboard.

Limitations

- **The prediction is a straight-line trend, not a failure model.** It answers "if the recent trend continues, when will this part reach 85 °C?". It cannot predict sudden failures. With no failure records in the data, no claim of prediction accuracy is made.
- **The data is historical (a fixed snapshot up to 30 April 2025)**, not a live stream. To include new logs, put them in the zip and re-run the pipeline.
- **Display resolution:** line charts with very many points are drawn from 5-minute (or longer) averages in the dashboard, and the exported per-server charts use 10-minute averages. Each block keeps its maximum, so no spike above 85 °C is hidden. All counts, tables and risk levels are calculated from the full-resolution data.
- **Box plots leave out readings of exactly 0 °C** (boards switched off). They are kept everywhere else.

Data-quality observations (reported, not hidden)

- 172.27.28.111 AUXTIN1 reads a constant 127 °C, which is probably a sensor fault (see 9.3).
- Some SDR sensors report exactly 0 °C for long stretches (for example SDR2 on 172.27.29.165 and SDR4 on 172.27.24.74). These are most likely boards that are switched off or disconnected. They are kept as logged and left out of the box plots only. They can lower average temperatures but can never cause a false overheating alert.
- FAN-6 reads 0 rpm all the time on several servers, which most likely means no fan is connected there. On 172.27.186.13 every fan reads 0 rpm (see 9.3).
- Server 172.27.186.50 has 11,627 minutes dated March 2023, most likely because of a wrong system clock. These rows are kept as logged.

12. How to reproduce the results

Requirements: Python 3.11 and Windows, macOS or Linux. PNG export also needs Google Chrome or Microsoft Edge installed. Run these from the project folder:

```
git clone https://github.com/KRISHNA2107DK/Server-Temperature-Monitoring-and-Predictive-Maintenance-Dashboard.git
cd Server-Temperature-Monitoring-and-Predictive-Maintenance-Dashboard
pip install -r requirements.txt
python src/pipeline.py          # ~4 minutes: rebuilds data/processed and results/tables
python src/export_charts.py     # rebuilds results/charts (HTML + PNG)
python -m pytest -q             # runs the unit tests
streamlit run src/dashboard.py  # opens the dashboard at http://localhost:8501
```

The pipeline is deterministic: running it again on the same zip file gives exactly the same tables. Library versions used: Python 3.11.9, pandas 3.0.2, plotly 7.1.0, streamlit 1.66.0, pyarrow 26.0.0, kaleido 1.5.0, pytest 9.1.1.

13. Project files and deliverables

Path (in the GitHub repository)	Contents
`src/pipeline.py`	Data loading, cleaning, one-minute summaries, overheating events, anomaly detection, trend, risk levels
`src/charts.py`	All chart designs (one place, so the dashboard and exported images look the same)

`src/dashboard.py`	The Streamlit dashboard (navigation, filters, six pages, downloads)
`src/export_charts.py`	Saves every chart as HTML and PNG
`tests/test_pipeline.py`	Unit tests
`results/tables/`	`server_summary.csv`, `component_risk_summary.csv`, `critical_breach_events.csv`, `anomalies.csv`
`results/charts/` and `results/charts/png/`	34 interactive HTML charts and 34 PNG images (overview charts, plus a trend chart and a cooling chart for each server)
`results/*.txt`	Logs of the pipeline run, chart export and test run
`data/processed/`	Pre-processed data used by the live dashboard
`SPEC.md`, `PLAN.md`, `README.md`	Requirements and acceptance criteria, implementation plan, setup instructions
`submission/`	This report (PDF and HTML) and the dashboard screenshots

14. Conclusion

The project meets every requirement of the problem statement. It is an interactive, publicly hosted dashboard that applies the 85 °C critical threshold to every component of all 14 servers. It presents the data in seven chart types, identifies overheating components and events, shows temperature trends, detects abnormal patterns automatically, and ranks components by maintenance risk using a transparent trend projection.

On the real data it found that **3 of the 14 servers** exceeded the critical limit. The most urgent case is **172.27.186.13**: motherboard sensors at up to 103 °C for most of the logged period, with no fan speed reported. It also found **172.27.28.111**'s hot SDR radio boards, and a likely faulty sensor reading a constant 127 °C. Every method, assumption and limitation is stated openly in this report, and every result can be reproduced from the public GitHub repository with the commands in Section 12.